

Excepción en clase JS

Generemos un ejemplo codificado en JS de excepción provocada por el usuario (ejemplo, no permitir una persona de menos de 18 años como objeto de una clase Persona).

Podemos crear una excepción manualmente con `throw` dentro del constructor de la clase `Persona`. Si la edad ingresada es menor que 18, el objeto no se crea.

</> JavaScript

```
class Persona {

  constructor(nombre, edad) {

    if (edad < 18) {
      throw new Error(
        "No se puede crear una persona menor de 18 años."
      );
    }

    this.nombre = nombre;
    this.edad = edad;
  }

  mostrarDatos() {
    console.log(`Nombre: ${this.nombre}`);
    console.log(`Edad: ${this.edad}`);
  }
}

// Datos ingresados por el usuario
const nombreIngresado = prompt("Ingrese el nombre:");
const edadIngresada = Number(prompt("Ingrese la edad:"));

try {

  const persona = new Persona(nombreIngresado, edadIngresada);

  persona.mostrarDatos();

  alert("La persona fue creada correctamente.");

} catch (error) {
```

```
console.error("Se produjo un error:");  
console.error(error.message);  
  
alert(error.message);  
}
```

¿Qué ocurre en el programa?

El usuario ingresa su nombre y edad:

</> JavaScript

```
const nombreIngresado = prompt("Ingrese el nombre:");  
const edadIngresada = Number(prompt("Ingrese la edad:"));
```

Luego se intenta crear el objeto:

</> JavaScript

```
const persona = new Persona(nombreIngresado, edadIngresada);
```

Dentro del constructor se valida la edad:

</> JavaScript

```
if (edad < 18) {  
    throw new Error(  
        "No se puede crear una persona menor de 18 años."  
    );  
}
```

La instrucción `throw` provoca una excepción. Cuando ocurre, JavaScript interrumpe la ejecución normal del constructor y pasa directamente al bloque `catch`:

</> JavaScript

```
catch (error) {  
    alert(error.message);  
}
```

Ejemplo válido

Si el usuario ingresa:

Nombre: Ana
Edad: 25

Se crea el objeto:

Nombre: Ana

Edad: 25

Ejemplo inválido

Si ingresa:

Nombre: Juan

Edad: 16

Se produce el error:

No se puede crear una persona menor de 18 años.

También conviene validar que la edad sea realmente un número:

</> JavaScript

```
class Persona {

  constructor(nombre, edad) {

    if (nombre.trim() === "") {
      throw new Error("El nombre no puede estar vacío.");
    }

    if (Number.isNaN(edad)) {
      throw new Error("La edad debe ser un número.");
    }

    if (edad < 18) {
      throw new Error(
        "No se puede crear una persona menor de 18 años."
      );
    }

    this.nombre = nombre;
    this.edad = edad;
  }

  mostrarDatos() {
    console.log(`Nombre: ${this.nombre}`);
    console.log(`Edad: ${this.edad}`);
  }
}

const nombreIngresado = prompt("Ingrese el nombre:");
const edadIngresada = Number(prompt("Ingrese la edad:"));
```

```
try {  
  
    const persona = new Persona(nombreIngresado, edadIngresada);  
  
    persona.mostrarDatos();  
  
    alert(  
        `Persona creada correctamente:\n` +  
        `Nombre: ${persona.nombre}\n` +  
        `Edad: ${persona.edad}`  
    );  
  
} catch (error) {  
  
    console.error(error);  
  
    alert(`Error: ${error.message}`);  
  
}
```

En este ejemplo la excepción es **provocada intencionalmente por el programa mediante throw**, debido a un dato inválido ingresado por el usuario.



devolveme el código completo en una pagina html sencilla

Aquí tienes un ejemplo completo en **una única página HTML**, ideal para mostrar en clase. El usuario ingresa el nombre y la edad, y si la edad es menor a 18 años se genera una excepción utilizando `throw new Error()`.

HTML

```
<!DOCTYPE html>  
<html lang="es">  
<head>  
    <meta charset="UTF-8">  
    <title>Excepciones en JavaScript</title>  
  
    <style>  
        body {  
            font-family: Arial, Helvetica, sans-serif;  
            margin: 40px;  
            background-color: #f4f4f4;  
        }  
  
        h1 {  
            color: #003366;  
        }  
  
        label {
```

Conceptos que se muestran en este ejemplo

- ✓ Clase `Persona`.
- ✓ Constructor con validaciones.
- ✓ Excepción provocada por el programador mediante `throw new Error()`.
- ✓ Manejo de excepciones con `try`.
- ✓ Captura de errores con `catch`.
- ✓ Uso de `finally`, que se ejecuta siempre.
- ✓ Creación de objetos a partir de datos ingresados por el usuario.
- ✓ Interfaz HTML sencilla para que el alumno vea el resultado inmediatamente en el navegador.

Es un ejemplo muy didáctico para introducir el concepto de **excepciones provocadas por el usuario** (o más precisamente, **excepciones lanzadas por el programa debido a datos inválidos ingresados por el usuario**).



incluime y validame también un setter para edad en este caso devolviendo el mismo código con este cambio

Este ejemplo queda incluso más orientado a POO, porque **la validación ya no está solamente en el constructor**, sino también en el **setter**. De esta forma, la regla de negocio ("una persona debe ser mayor o igual a 18 años") se cumple tanto al crear el objeto como al modificar la edad posteriormente.

HTML

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Excepciones en JavaScript</title>

  <style>
    body{
      font-family: Arial, Helvetica, sans-serif;
      margin:40px;
      background:#f4f4f4;
    }

    input{
      padding:8px;
      margin:5px;
      width:220px;
    }
  </style>
</head>
<body>
```

Explicación para los alumnos

En este ejemplo, la validación se realiza **dos veces utilizando el mismo método**:

```
</> JavaScript
```

```
this.setEdad(edad);
```

El constructor **no asigna directamente**:

```
</> JavaScript
```

```
this.edad = edad;
```

sino que llama al **setter**, el cual verifica que:

- la edad sea un número;
- la persona tenga al menos 18 años.

De esta manera, si más adelante alguien intenta modificar la edad:

```
</> JavaScript
```

```
persona.setEdad(30);
```

el cambio será válido.

En cambio:

```
</> JavaScript
```

```
persona.setEdad(15);
```

provocará una excepción:

```
Error: No se puede crear una persona menor de 18 años.
```

Este diseño es muy recomendable en Programación Orientada a Objetos porque **centraliza las validaciones en un único lugar**, evitando duplicar código y asegurando que la regla de negocio se cumpla tanto durante la creación del objeto como en cualquier modificación posterior.

